VIGILFOIL: Workload- and Response-Aware Evidence Accounting for Budgeted Security Monitoring in RPL Networks

Sharad Kumar Dubey, Vasim Khan
Department of Computer Science and Engineering
IIITDM Kancheepuram

***Abstract*: A rise in CPU activity does not, by itself, distinguish an attack from a legitimate job. A controller's own repair actions can also change the traffic it observes. We address both sources of ambiguity in VIGILFOIL, a gateway-based security controller for RPL networks. Workload permissions help it estimate normal resource use. It keeps original and repaired delivery apart, accepting an issued repair lease as an explanation for missing original packets only when the records support it. Host warnings guide queries and helper selection within a payload budget. We tested the accounting rules in native Cooja, then evaluated the combined system in 42 runs with ten instruction-emulated Z1 motes. Response accounting cut defense payload by 17.97% in the native comparison. In the combined experiment, mean offered-work delivery was 98.37%; the network-only control reached 98.03% with the same instrumentation. The mean paired payload cost rose by 1.38%. All four configured host-misuse episodes were detected, and seven of ten sparse bursts were covered. False warnings remained a problem. In complete benign runs, 15.53% of scored observations triggered warnings. The accounting rules reduced unnecessary follow-up traffic, but the host detector still struggled with benign activity after integration.**

***Index Terms*: Internet of Things, RPL, host-oriented mimicry, intrusion detection, evidence accounting, workload context, bounded recovery.**

I. Introduction

Normal packet traffic can continue while an IoT node performs unauthorized work. Host-oriented mimicry attacks (HMAs) exploit that overlap. CIPHER [1] uses collaboration among IDS nodes to detect such attacks. Its placement and scheduling rules keep collaborators available while avoiding continuous execution of every IDS. We work under the same resource constraint. Our question concerns the observations that lead a controller to investigate further.

Consider a node that receives a legitimate computation job. Its CPU usage rises. Later, the controller asks it to send some packets through a helper, so fewer original packets reach the collector. Both changes are authorized. A detector that lacks the job permission may flag the rise in CPU use. A controller that overlooks its redirect may ask the node to explain packets that the controller itself moved.

Repair can hide a failure, too. Even if a helper delivers every item, the original path may still be faulty. We therefore keep the cause and route of an observation when the available records support them. We ask whether doing so changes communication cost and service outcomes under limited observation and repair budgets.

The gateway runs VIGILFOIL's inference and policy logic. Motes measure their resources, report application work, and carry out forwarding instructions. A host warning raises query priority and makes the warned node ineligible to help another source. The node stays connected. This lets us measure how host evidence affects network response, although the gateway has no mechanism for stopping a malicious process.

We link the controller's decisions to records of authorized work and repair. The checks distinguish submitted work, original receipts, and delivery through an authorized helper. Separate workload permissions help explain resource changes before the controller requests more evidence. We test this link in closed-loop simulations, removing components and comparing simple controls. The contribution lies in how these records constrain the next decision. Recurrent models, authentication, and repair themselves are established techniques.

II. Related Work and Positioning

A. CIPHER and collaborative host monitoring

CIPHER [1] uses IDS placement and scheduling to keep collaborators available. It combines host/network detection with a lightweight behavior monitor. CONTRAST [2] studies adaptive scheduling for host-based mimicry detection, while LITMUS [3] addresses collaboration when some collaborators are dishonest. These are the works closest to our problem. They bring together host evidence, temporal models, collaboration, and trust.

We also use both host and network observations. We wrote the ledger, gateway transport, and repair controller. For the comparisons, we reconstructed the published detector components and the framework rules described in enough detail to implement. We did not have the authors' complete executable systems. Table I records the scope and limits of each reconstruction.

Scheduling decides when an IDS runs. The accounting rules answer a later question: what does an observation mean after the controller has acted? A valid redirect can reduce the expected original count. A repair receipt, however, cannot establish that the original path is healthy. This distinction is what we test. Context-aware detection and collaboration are not new in themselves.

B. Routing response and resource-constrained decisions

SRF-IDS [4] uses trust to guide routing, and we built a response baseline from its source release [5]. Abbas et al. [6]

study IDS scheduling under resource limits. Work on quickest-change detection [7] balances observation cost against delay. GRU-D [8] uses missingness and time gaps in recurrent prediction. These studies inform the budget and temporal-state questions here, although we have no corresponding optimality result for VIGILFOIL's fixed priority rule.

A response can hurt the very traffic it is supposed to protect. We tested this in native Cooja by putting the suspect relay on a bridge that legitimate sources depend on (Section V-D). With monitoring only, offered delivery was 91.88%. With guarded exclusion it dropped to 58.41%. When we required confirmation before exclusion, it recovered to 84.47%. These numbers come from our own controller experiment. They are what pushed us toward limited repair instead of disconnection based on a host warning alone.

C. Temporal models and broader comparisons

Sun, Johnsson, and Banerjee [9] add TX/RX features to RPL statistics for LSTM detection. Lunderbye et al. [10] apply a foundation model to RPL attack classification. We used their released data and the official MOMENT implementation [11] for a separate routing benchmark. It provides a recent temporal-model comparison. The routing labels do not identify hidden host work.

No single dataset supplies all the observations we need. We therefore use HMA reconstructions to compare detectors, an SRF source port to examine routing response, and public CSVs to test routing classification. The live substitutions receive the same observations and use our controller. Their delivery results can be compared within that setting, but they do not rank the complete published systems.

III. Network, Threat Model, and Evidence

A. Network and execution boundary

We work with a low-power IPv6 network running RPL [13]. A radio root collects application packets and reports from sources and relays. The gateway has more compute power and storage than any mote, and it keeps the evidence ledger. For the final integrated experiment we use ten Z1 motes, the root included, under instruction emulation in Cooja [14]. The unit-disk radio model gives a 30 m transmission range and a 45 m interference range. Transmit success is set to one. Reception success is 0.90 in the benign context, one for mixed attacks, and 0.85 for sparse/loss attacks. All motes run the same application and protocol firmware. The gateway runs on the host computer.

Packet loss and limited mote memory restrict what the gateway can observe. Queries and repair packets use network capacity as well, so we include them in the traffic count. Mandatory reports are counted separately from the query/repair reservation. The reservation is therefore not a cap on total traffic. Nor can payload bytes alone establish radio energy use.

B. Adversary and trusted components

The adversary we evaluate runs unauthorized arithmetic work and may selectively drop packets in transit. Sparse misuse takes the form of brief, recurring CPU bursts. Earlier experiments also cover control forgery, report loss, dishonest forwarding claims, and stronger helper dropping. The evaluator knows the attack configuration. Oracle events are not fed to the gateway.

We trust the root, the gateway, the workload issuer, and the resource monitor. The attacker can misuse the application or drop packets but cannot swap out a trusted monitor's counters in the main experiment. A valid tag proves who sent a report. It does not prove that the measurement is true. We assume a benign startup when building the initial resource reference. Compromised roots, poisoned startup, and hardware attestation are all outside our scope. The keys in the simulations are lab fixtures.

The three-Z1 task experiment illustrates the limit of inferring intent (Section V-D). We ran equal amounts of unannounced benign work and hidden malicious work. Each seed gave 55 identical result observations for the matched pair. The gateway had no basis for separating them. Here, "unexplained" means that the available permissions and observations do not account for the activity. It does not establish malice. Stale or missing evidence stays unknown.

C. Work and receipt semantics

For source i and a recently closed epoch e, the ledger keeps offered work O, submitted work U, blocked work B, original attempts P, repair attempts R, redirected work D, and lease identity ℓ. The source report must satisfy

$$O_{i,e} = U_{i,e} + B_{i,e}. \tag{1}$$

We check each report for count consistency, freshness, sequence, nonce, and action scope. Work identifiers let us match receipts without mistaking multiple copies for multiple completed items. We write the original receipt set as $\mathcal{Y}^o$ and the repaired set as $\mathcal{Y}^r$. Their union tells us how much service was completed. Only $\mathcal{Y}^o$ says anything directly about the original path.

We accept a lower original count only when root-issued lease history backs it up. In compact notation, the expected original work is

$$E^o_{i,e} = \begin{cases} P_{i,e}, & \text{validated redirection supports } P_{i,e}, \\ U_{i,e}, & \text{otherwise.} \end{cases} \tag{2}$$

The unexplained original deficit is $d_{i,e} = \max(0, E^o_{i,e} - |\mathcal{Y}^o_{i,e}|)$. A redirection claim with no backing cannot shrink this expectation. And missing evidence does not mean the deficit is zero or the source is benign.

Suppose a source submits ten items and a valid lease redirects four. Six original attempts remain. If six original receipts and four repair receipts arrive, all ten items have been delivered. Expecting ten original receipts would create four deficits caused by the controller's own instruction. Looking only at the union would hide whether repair was needed. Both records are needed to tell those cases apart. This is an illustration, not a measured run.

TABLE I
BASELINE PROVENANCE AND THE SCOPE OF OUR COMPARISONS. ALL RESULTS REPORTED IN THIS PAPER ARE OUR MEASUREMENTS UNLESS EXPLICITLY ATTRIBUTED TO A SOURCE.

Method	Implemented or reused elements	Boundary of the comparison
CIPHER [1]	Correlation-grouped LSTM/FC autoencoder, monitor, and reconstructed scheduling/placement replay	Author implementation unavailable. Widths and some training/aggregation choices are reconstructed. Latest live panel substitutes a detector into our controller.
CONTRAST [2]	CPU/TX autoencoder and scheduling/aging reconstruction	Width and attack-threshold sensitivities are explicit choices. Sampling and deployment differ from the paper.
LITMUS [3]	Host AE, network ensemble, Bloom consistency, trust replay	Some training and forgetting details are unavailable. Native substitution uses network features, not the original full framework.
SRF-IDS [4], [5]	Archived detector, trust, messages, and routing objective	Native Cooja/UDGM port replaces Whitefield/NS3. ABI, addressing, capacities, and receive-path behavior required declared adaptations.
Radio LSTM [9], [12]	Pinned author class; hidden width 10; source-trained controls	Released data contain 14 RPL fields, not the full 18-feature radio view. Our measured-radio panel is a separate reconstruction.
MOMENT RPL [10], [11]	Official frozen pretrained encoder and trained linear head	Public-data experiment uses 14 channels; paper parameter counts imply nine. No claim of MCU execution or full-model live HMA response.

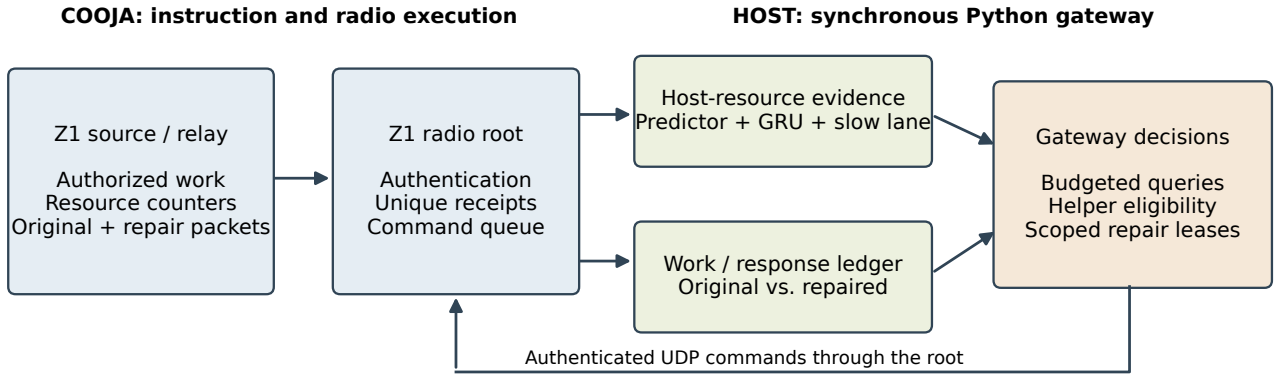


Fig. 1. Implemented evidence and action flow. Mote execution and packet transport occur in Cooja; inference and decision logic run in the Python gateway. Original and repaired receipts remain distinct. Ground-truth attack logs are used only by the evaluator.

IV. COMBINED DECISION METHOD

A. Workload-conditioned resource evidence

Each mote reports cumulative CPU, TX, and RX ticks, RPL activity, own/transit counters, source time, and sequence. The root checks the 84-byte host report and rejects a wrong session or a sequence that does not increase. The gateway computes causal differences from the reports it receives. Root-authorized work has separate amount and expiry limits. The permission comes from the root; it is not inferred from the anomaly it is supposed to explain.

For measured CPU activity c_t, network context x_t, and authorized work w_t, the benign predictor is

$$\widehat{c}_t = f_{\text{net}}(x_t) + \alpha w_t, \tag{3}$$

$$r_t = c_t - \widehat{c}_t. \tag{4}$$

We fit the network predictor and the coefficient on benign source deployments. CPU activity is the increment in active ticks divided by elapsed source time and 32,768. The final coefficient is $\alpha = 3.219255 \times 10^{-5}$ in the implementation's activity-per-work units. TX and RX expectations are predicted too. Nine discriminator inputs capture observation timing/gaps, signed CPU/TX/RX residuals, and relative residuals. Source identities index state and attribution; they are not used as numerical predictors.

The host head uses a 16-unit GRU over eight received observations. We average its score with that of a histogram-gradient-boosting classifier. The neural lane triggers when at least three of the last five scores exceed its threshold. We first trained the head in the standalone Z1 experiment described in Section V-D. For integration, we retained those weights and refitted benign resource prediction and calibration. Python training used ten Z1 source runs with 21,331 rows and 20,701 eight-observation windows. Six development runs supplied a further 12,353 windows. We used class-weighted binary cross-entropy and AdamW at 0.001. Weight decay was 0.01 and the batch size was 256. Gradients were clipped at one. Of the checkpoints from 100 epochs, we retained the one with the lowest development loss. The histogram classifier used 150 iterations and at most 15 leaf nodes, with a learning rate of 0.05 and L2 regularization of three. The head thus uses supervision that the benign-only comparison autoencoders do not receive.

B. Slow resource evidence

A second lane watches for moderate unexplained CPU activity that persists across observations. Let σ_t be a noise scale that depends on forwarding rate, with a relative expected-CPU floor. Let b_i be a capped startup reference, and Δt the gap in received source time. The implementation works as follows:

$$z_t = \mathrm{clip}(r_t/\sigma_t - b_i, -3, 5), \tag{5}$$

$$S_t = \min\left(2h, \max\left(0, e^{-\widetilde{\Delta t}/120} S_{t-1} + (z_t - 1)\frac{\widetilde{\Delta t}}{5}\right)\right), \tag{6}$$

where $\widetilde{\Delta t} = \min(15, \max(1, \Delta t))$ seconds and h is the calibrated threshold. After an eight-observation warm-up, the first 20 residuals set the capped reference. That reference is the median standardized residual, clipped to $[-2, 2]$. This recurrence is an engineering rule. We make no claim about detection-delay optimality.

A warning from either lane lasts 30 seconds. We set both thresholds on complete benign calibration runs, placing them above the calibration maxima with fixed margins. The frozen values are 0.103122 for the neural score and 21.118909 for slow evidence. The slow accumulator decays with a 120-second time constant and is capped at twice its threshold. This calibration controls what we saw on the development set. It does not guarantee the same false-warning rate on a new deployment. A poisoned startup can corrupt the reference.

C. Evidence acquisition and repair

Every ten seconds, the gateway ranks sources using

$$\pi_i = 10d_i + 20a_i + \mathbf{1}(n_i \wedge m_i), \tag{7}$$

where a_i is a current host warning, n_i is the network-priority alarm, and m_i flags missing current work evidence. This is a fixed policy rule, not a learned scheduler. A host warning can trigger investigation even when delivery looks fine. Repair needs both a delivery deficit and source-work evidence.

We only consider a helper if the source lists it as a neighbor. The helper's recent route must reach the root without a cycle and without looping back through the source. We reject any candidate that has a large observed forwarding deficit or a current host warning. This is a check based on what we have observed. It is not proof that the helper is honest. Even a node with a valid lease can refuse to forward.

The controller looks first at the epoch ending two decision intervals ago. It can also pull support from either of the two epochs before that. Without a source-work fact, the controller falls back to a two-item default for investigation priority but keeps the unknown flag. It cannot authorize repair from that default. Work reports expire after 35 seconds. Query replies expire after 20 seconds. A repair lease lasts ten seconds.

At each decision, the gateway reserves at most 480 application bytes for discretionary work. It reserves 136 bytes for a query/reply and 248 bytes for a two-copy repair lease. The latter comprises 120 control bytes, 72 source-copy bytes, and 56 helper-copy bytes. The root, source, and helper check authentication, nonce, scope, and quota. We charge mandatory reports and workload permissions separately. The reservation limits admitted queries and repair, rather than all security traffic.

D. What the response accomplishes

The gateway sends each selected instruction to the root queue. The root transmits it over UDP, and the receiving firmware carries out the redirect or helper forward. We then measure what actually gets delivered. If the host is doing unauthorized CPU work, that work can keep going after the warning; this implementation has no way to kill a process. What the response does is investigate evidence and try to recover network delivery.

V. Implementation and Experimental Protocol

A. Implementation details

The final root and leaf firmware use 6,294 and 5,782 bytes of static RAM, respectively. Those numbers leave out worst-case stack and heap. A full source-work report is 108 bytes. An original application packet is 28 bytes. A source repair packet is 36 bytes. A helper forward is 28 bytes. The root has a four-entry command queue, polled every 125 ms. Software AES is verified against a known test vector. Saved packet evidence goes through an independent check for authentication and command consistency.

The simulation bridge passes root records and command bytes to the gateway synchronously. Root polling and packet delays advance simulated time; gateway inference pauses it. Because of that, host inference timing is not part of the simulated end-to-end latency. We use no FIT IoT-LAB hardware. We make no MCU inference or real-time qualification claim.

B. Experimental tracks and units

Table II groups the experiments by question and execution environment. Native Cooja executes the network and controller code without emulating MCU instructions. Z1 runs do emulate those instructions. Python replay and public-data fits answer other questions. Neither measures the outcome of a live response.

C. Combined-system split and scenarios

Source deployments 1111 and 1112 supply benign resource data and autoencoder fitting. Deployments 1113 and 1114 are for calibration on benign loss and legitimate workload changes. Together they give us 2,276 source rows and 2,241 calibration rows. We kept the earlier supervised GRU/classifier weights. Preprocessing and thresholds for the new rig use only source/calibration data. Final seeds 1151 and 1152 were held out until the model was frozen.

A final run lasts 900 seconds. Sources offer application work from 180 to 840 s, receive authorization for work from 300 to 780 s, and face configured attacks from 400 to 700 s. Seed 1151 uses the bypass layout with target 2; seed 1152 uses the grid layout with target 3. Each deployment covers three contexts. The first has authorized activity and benign

TABLE II
EXPERIMENTAL TRACKS AND THEIR EXECUTION ENVIRONMENTS. COUNTS ARE NOT INTERCHANGEABLE INDEPENDENT SAMPLE SIZES.

Track	Environment and extent	Question and unit of evidence
Controller accounting	Native Cooja; 887 accepted candidate runs, including five bring-up smokes	Main accounting contrast: 36 matched cases per variant, three deployment seeds. Scale, identity, nested-component, and stronger-helper panels remain separate.
Live model substitutions	Native Cooja with exported C inference; 126 runs	Seven detectors, 18 runs each, common 15 root-visible features and controller; CPU absent.
Original-feature observations	Instruction-emulated Z1; 58 observation runs, followed by Python fits/replay	Includes two-hour benign sources and ten-hour attack runs; replay scheduling is not on-mote neural execution.
Public routing benchmark	Released Cooja CSVs; Python training/inference	960 selected runs from 1,200 public files; common five-class N20 subset uses matched 14-channel inputs and duplicate exclusions.
SRF source port	Native Cooja/UDGM; 72 one-hour runs	Blackhole subset, 5/10/15 detectors; original source with explicit platform and receive-path adaptations.
Combined system	Ten emulated Z1 motes plus Python gateway	Four source/calibration runs and 42 final 900-second runs; seven policies, three contexts, two deployment blocks.

disturbance. The second adds sustained 250-round hidden CPU work and selective forwarding. The third has 1,000-round CPU bursts, each lasting four seconds every 60 seconds. Legitimate traffic changes and selective forwarding occur alongside those bursts, with nominal reception set to 0.85. Each topology has only one seed. Their effects cannot be separated, so these two blocks do not establish general topology robustness.

All seven policies share the transport and instrumentation. We compare the combined controller with a network-only control, CIPHER-style and CONTRAST-style autoencoders, and three component removals. The removals disable workload context, action accounting, and slow evidence, respectively. The network-only control ignores host warnings but retains shadow host computation and telemetry. This isolates the effect of using the warnings. It does not measure the lowest possible cost of a controller with host monitoring removed.

For the CIPHER-style adaptation: correlation-grouped width-two LSTMs over 18 available fields, window 16, 250 epochs, batch 32, Adam learning rate 0.05. For the CONTRAST-style adaptation: CPU/TX inputs, width eight, window 32, 200 epochs, batch 256, learning rate 0.001. Both are trained on benign new sources and calibrate three-of-five warnings on complete benign runs. These short fits and the schemas we had available differ from the original distributed experiments. Component removals keep the full model's frozen weights and thresholds. They test whether the model depends on each component, not what a separately optimized replacement would achieve.

D. Supporting experiments and controls

Before combining the host and network controllers, we ran four smaller experiments to test our assumptions. We conducted all of them ourselves. They answer specific design questions, not add more independent replications of the final comparison.

For witness sampling, we ran 900-second native Cooja simulations on a ten-node RPL chain. Sender 4 transmitted through suspect relay 3 to honest witness 2. The relay dropped 50% of application packets during attack episodes. We varied legitimate bursts, reception loss, and an evader that could spot marked probes. Fixed 20-second probing and root-triggered probing with a mandatory 40-second audit used the same monitored segment. We scored 60-second histories with an eight-second acknowledgment deadline and benign-only calibration. In a follow-up, we swapped dedicated probes for ordinary application data and compared four approaches: deterministic every-fourth-packet acknowledgments, keyed 25% sampling, authenticated receipts, and root delivery counts. That follow-up used four test seeds and separate benign calibration seeds. These designs test observation availability and limited evasion, not arbitrary witness compromise.

For response safety, we compared three options in native Cooja: monitoring alone, temporary parent exclusion with restoration, and confirmation before exclusion. We varied whether legitimate paths depended on the suspect relay. Offered work stayed in the service denominator even when exclusion made its destination unreachable. The point of this design is to expose collateral loss at bridge relays. Treating reduced attacker traffic as a service gain would miss that entirely.

For task integrity, we used ten-node, 900-second native Cooja runs. The gateway sent two eight-field jobs every ten seconds and knew the intended computation. Paired full-checking policies either left missing results unresolved or completed them locally, capped at 16 item computations per ten seconds. We checked correctness and completion within 30 seconds. Eight more 900-second three-Z1 instruction-emulated runs tested the signed arithmetic protocol and matched the benign/malicious workload ambiguity. This task contract is a separate extension. It is not a component of VIGILFOIL's final combined model.

We developed the host head using CPU/TX/RX and workload observations from instruction-emulated Z1 runs. Training is described in Section IV-A. We tested fresh misuse episodes,

legitimate activity, and intermittent bursts before transferring the head to the combined experiment in Section V-C. There, we refitted its benign predictor and thresholds. The two experiments therefore differ in both traffic and calibration.

E. Metrics and validation

Offered-work delivery is

$$\mathrm{PDR}_O = \frac{|\mathcal{Y}^o \cup \mathcal{Y}^r|}{\sum_{i,e} O_{i,e}}, \tag{8}$$

with unique genuine identifiers and blocked/unreachable offers kept in. Worst-source delivery takes the minimum source fraction within a run and then averages across runs. Defense cost sums up the attempted application payload for mandatory reports, authorization, queries/replies, grants, and repair copies. Retransmissions, PHY headers, and Joules are not derived from that sum.

We report host detection at several levels. Episode hits and burst coverage describe whether warnings occur during attacks. Observation precision/recall/F1 and the pooled negative-observation false-positive rate (FPR) describe individual scored observations. We also count warnings in entirely benign runs. A sparse burst is covered if a warning occurs during execution or within 15 source-seconds of its last execution. An earlier warning can provide coverage without a new detection. For onset delay, we use the first actual hidden-work event from the independent oracle log, rather than relying on the configured start time.

We check that simulations finish and that causal features are finite. We verify source-only fitting, match offers to receipts, and check command authentication and reservation admission. A saved-model replay checks the recorded actions independently. The model hash was frozen before the final runs; firmware hashes were checked for each run. The gateway source hash was recorded after completion. Replay reproduced the actions, but it does not replace a pre-run source record.

VI. Results

A. Response accounting reduces conditional protocol cost

Table III reports the main native-controller panel: ten nodes, bypass/grid layouts, seeds 521–523, and six contexts covering activity, benign loss, gray dropping, burst dropping, probe-reactive dropping, and forgery. Without action accounting, the controller uses 30,424.6 bytes/run. With full accounting, it uses 24,958.3. That is a 17.97% drop; about 5,466 bytes per run. An exploratory bootstrap over the three deployment-seed blocks puts the extra cost of the unaware variant at 5,059.7 to 5,779.7 bytes. With only three blocks, though, this interval should not be read as a deployment-wide guarantee. Delivery differs by just 0.057 percentage points, and the uncertainty spans both benefit and harm.

Removing context also drives up payload in Table III. To look at ordering specifically, we ran another 18-case comparison where only priority ordering was turned off. Context, authentication, reporting, and redirect settings stayed the same. The full controller used 41.61% less payload. We also repeated the action-accounting test in an earlier native implementation with a main panel and a mesh holdout. The savings there were 13.13% and 28.46%, and delivery shifted by $+0.190$ and -0.168 percentage points. So the cost effect keeps showing up across setups. The delivery effect, however, is not always positive. We keep the cohorts separate.

TABLE III
Native Cooja controller ablations, 36 cases per row. Delivery is offered-work delivery; payload is bytes/run.

Variant	Delivery (%)	Worst (%)	Payload (B)
Full controller	97.525	93.350	24,958.3
Action accounting off	97.468	93.178	30,424.6
Without context	97.449	93.054	33,190.3
Model influence off	97.543	93.335	23,570.7
Without repair	97.342	92.601	24,815.7
Auth. checks off	96.486	84.007	27,067.6
Periodic (20 s)	97.819	93.636	32,076.0
Budget 136 B	97.347	92.595	22,645.8
Budget 960 B	97.584	93.329	29,293.0

Periodic reporting is a strong alternative. It reaches 97.819% delivery and better mean worst-source delivery, but at 7,117.7 more bytes per run. Turning off the learned priority model saves 1,387.7 bytes/run without a clear service loss. So we cannot pin the protocol benefit on learning alone. Coverage gaps are also real: 88.53% of ledger states in the full native controller stay unknown. Every audited strict explanation has a supporting causal direct receipt under our source-honesty assumptions. But that check does not clear up the unknown states.

B. Latest combined architecture

The combined system reaches 98.37% mean delivery in Table IV, compared with 98.03% for the equally instrumented network-only control. The mean paired difference is 0.336 percentage points. Across individual cells it ranges from -0.600 to $+1.263$, so the gain is not consistent everywhere. Mean worst-source delivery rises from 91.92% to 94.19%, while mean paired defense payload rises by 1.38%. These are descriptive results from two blocks.

The combined controller fires a warning during all four configured host-misuse episodes. It catches seven of ten sparse bursts. Table V shows the first-execution delays and how many packets were actually dropped. In the grid runs, the relay drops no packets because it has no transit traffic to attack. We still keep those runs as host-misuse tests. Real forwarding exposure happens in the bypass layout: 12 drops in the mixed case, three in the sparse case.

Figure 2 keeps each matched cell visible instead of burying variation behind the mean. The worst-source annotations show whether a shift in average service also reaches the source that gets the least.

C. Sensitivity is accompanied by false warnings

At the frozen threshold, pooled host precision is 21.61%, recall is 78.21%, and F1 is 0.339 (Table VI). The pooled negative-observation FPR is 11.03%. In the two complete benign runs, warnings occur on 322 of 2,074 scored observations,

TABLE IV
CLOSED-LOOP COMPARISON AND ABLATION IN COOJA WITH TEN INSTRUCTION-EMULATED Z1 MOTES AND THE PYTHON GATEWAY. EACH ROW CONTAINS THE SAME SIX CONTEXT/DEPLOYMENT CELLS. HOST OBSERVATIONS ARE CORRELATED.

Policy	Delivery (%)	Worst-source (%)	Defense B/run	Episodes	Benign warnings/scored	Sparse coverage
Combined	98.37	94.19	133,901	4/4	322/2074	7/10
Network-only (instrumented)	98.03	91.92	132,095	–	–	–
CIPHER-style AE	98.01	92.43	132,137	0/4	58/1936	0/10
CONTRAST-style AE	97.93	92.35	132,261	0/4	40/1636	0/10
Without workload context	97.63	90.31	135,553	4/4	380/2090	10/10
Action accounting off	98.42	94.99	134,725	4/4	326/2083	7/10
Without slow evidence	98.24	92.94	132,802	1/4	195/2068	3/10

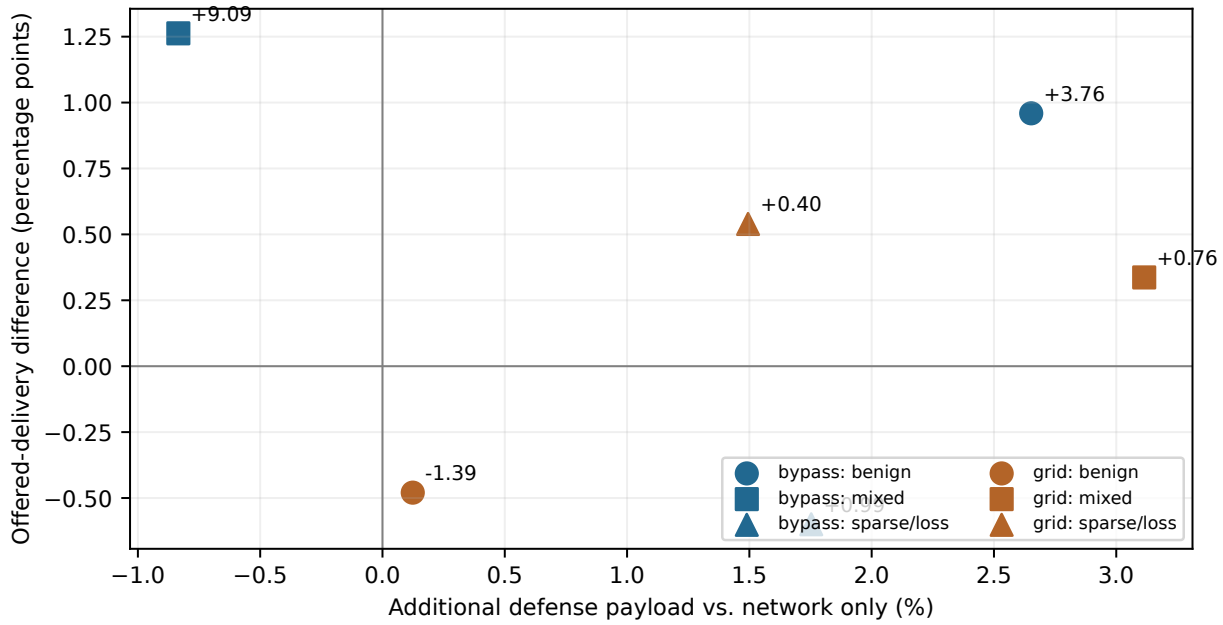


Fig. 2. Paired combined-versus-network-only outcomes for all six final Z1/gateway cells. Marker shape identifies context and color identifies deployment. Point annotations give the worst-source delivery difference in percentage points. Rightward movement costs more payload; upward movement improves offered delivery. The two deployment blocks are not independent topology replications.

TABLE V
FULL COMBINED MODEL: ACTUAL ATTACK EXPOSURE AND ONSET DELAY (EMULATED Z1 PLUS GATEWAY).

Layout	Context	Drops	Delay (s)	Bursts
Bypass	Mixed	12	41.09	–
Bypass	Sparse/loss	3	36.19	4/5
Grid	Mixed	0	18.17	–
Grid	Sparse/loss	0	129.14	3/5

TABLE VI
POOLED HOST-OBSERVATION METRICS IN THE FINAL Z1/GATEWAY STUDY. FPR USES ALL LABELED NEGATIVE OBSERVATIONS.

Policy	Prec. (%)	Rec. (%)	F1	FPR (%)
Combined	21.61	78.21	.339	11.03
CIPHER-style	0.00	0.00	.000	1.85
CONTRAST-style	0.00	0.00	.000	1.80
Without work	22.77	100.00	.371	13.17
Without action	20.72	78.21	.328	11.62
Without slow evidence	8.52	12.88	.103	5.35

or 15.53%. The rates use different denominators. The pooled rate also includes labeled negative observations from attack runs and recovery periods. Neither counts independent alert incidents per hour.

Take away the slow accumulator and only one episode gets hit, with three sparse bursts covered. Benign warnings drop from 322 to 195. It is a straight trade-off between sensitivity and false warnings. The GRU is still there in this removal, so the result is about the added accumulator, not about recurrent memory in general.

Workload permissions improve mean delivery and reduce benign-run warnings from 18.18% to 15.53%. Yet removing that information gives a higher F1 of 0.371 and covers all ten bursts. The onset records explain why this needs care. In all four attack runs, a warning was already present within 15 seconds before hidden work began. Two sparse onset differences are negative. These warnings carried over; they did not anticipate the attack.

Neither autoencoder substitution warns during any of the four host episodes at its frozen threshold. Both produce fewer benign warnings than our combined detector. They train on benign data only, while our head uses attack labels and a separate residual accumulator. The common controller lets us isolate what each detector does in this setup. But it does not erase the differences in supervision, and it does not prove that the original CIPHER and CONTRAST frameworks are worse.

Figure 3 makes the sensitivity cost easy to see. Removing slow evidence pulls down both warnings and coverage. Removing work context pushes up coverage and benign warnings together. These points do not trace out an optimal operating frontier.

D. Telemetry changes the cost balance

Action accounting saves less payload after integration. Removing it raises mean paired payload by 0.61% and raises delivery slightly to 98.42%. The 17.97% saving in Section VI-A belongs to the native experiment. In the six full combined runs, host reports contribute 600,264 of the 803,404 defense bytes, or 74.715%. Mandatory work reports add 128,304 bytes. Most of the cost is therefore in periodic reporting (Figure 4). A limit on queries and repair leaves that cost largely intact.

The saved bundle occupies 1,432,961 bytes, including the comparison models. The predictor accounts for 1,173,474 bytes, the classifier for 227,415 bytes, and the GRU state for 7,203 bytes. Although the GRU has only 1,313 parameters, it is only part of the gateway artifact. We also replayed the same inputs in Python. Across 203 scored host reports, median processing time was 5.13 ms and the 95th percentile was 5.53 ms. These are host timings, not MCU timings. Likewise, the complete artifact and a published detector-weight footprint cover different memory boundaries.

Figure 5 reveals which ablation effects hold up across conditions. The averages in Table IV should be read next to these cells. Neither the full model nor any single removal wins in every context.

E. Broader live and public-data comparisons

In the native live-substitution experiment summarized in Section V-B, seven detectors use the same source-work, query, and repair controller. Each receives the same 15 root-visible features. CPU is absent, so the CONTRAST-style substitution uses TX/RX proxies. Table VII reports the response outcomes measured in the running network. We do not infer delivery from offline detector scores. The LITMUS-style ensemble and frozen SGD controls are competitive, while the CIPHER-style adaptation uses more payload. This ranking applies to the tested inputs and supervision, not to the original frameworks as a whole.

The public-data benchmark uses released RPL CSVs [12] and Python training. We use 14 channels from 20-node runs with five classes. The window is 100 and the stride is 16. We found five exact-content duplicate groups across splits. In the accepted rerun, we excluded three validation files and two test files; the training copies remain disclosed. This leaves effective MOENT validation/test counts of 47/48. We split complete runs. Overlapping windows from one run are not randomly shared between training and testing.

TABLE VII
SEPARATE NATIVE COOJA LIVE DETECTOR SUBSTITUTIONS, 18 RUNS PER ROW. COMMON CONTROLLER AND 15 ROOT FEATURES.

Detector	Delivery (%)	Defense B/run
CIPHER-style AE	97.671	39,053.8
CONTRAST-style AE	97.830	27,862.0
LITMUS-style ensemble	97.887	27,941.3
Radio-LSTM architecture	97.872	26,088.0
Frozen SGD, fixed threshold	97.877	25,613.8
Frozen SGD, calibrated	97.844	25,876.4
Source-refitted SGD	97.741	30,448.7

TABLE VIII
SEPARATE PUBLIC RPL CLASSIFICATION BENCHMARK (PYTHON ON RELEASED COOJA CSVS). THESE ARE ROUTING-CLASSIFICATION RESULTS, NOT THE COMBINED MODEL'S HMA RESULTS.

Model	Macro F1	Fit-seed range
Official MOMENT-small + head	.8766	.8747–.8804
LSTM, hidden width 10	.5832	.5209–.6866
LSTM, current only	.4839	.4278–.5123
SGD with history	.7294	.7233–.7343
SGD, current only	.4667	.4328–.4882
Random forest, summaries	.7597	.7589–.7611
Dynamic INT8 LSTM	.4382	.4164–.4592

MOMENT has the highest score in this benchmark. Its 14-channel adaptation, however, contains 35,373,253 parameters, occupying 141,493,012 FP32 parameter bytes. A separate nine-channel smoke test matches the source paper's parameter boundary. The classification result alone does not establish feasibility on a mote. Temporal SGD and random forests over summaries also outperform the small LSTM. With dynamic INT8, that LSTM's F1 falls from 0.5832 to 0.4382. Median host inference time rises from 0.188 to 1.12 ms. Compression did not help in this configuration.

To test added features, we collected a separate Z1 dataset. Training uses seeds 561–563, validation uses seed 564, and testing uses seeds 565–566. The collector sends CPU/TX/RX and work counters in 72-byte telemetry. We form causal statistics over 60 seconds, updating them every five seconds. The input sets contain 14/18/20/26 fields. Each set adds fields to the preceding one. We fit them using source data only. In Python replay, LSTM F1 is 0.5449 with 14 RPL fields. It falls to 0.4261 with radio features, reaches 0.5086 when CPU is added, and is 0.5030 after adding work/freshness. A supervised CPU-augmented SGD reaches 0.7062. More context does not, by itself, improve the model. Its value depends on how the model uses it and how well it transfers. This collector uses plain telemetry, not the final authenticated combined protocol.

F. Original-feature replay and the SRF source port

The long-duration Z1 observations show why we keep framework replay separate from detector substitution. Four ten-hour grid runs execute hidden work or staged false readings/file writes. In our collection, however, the compromised

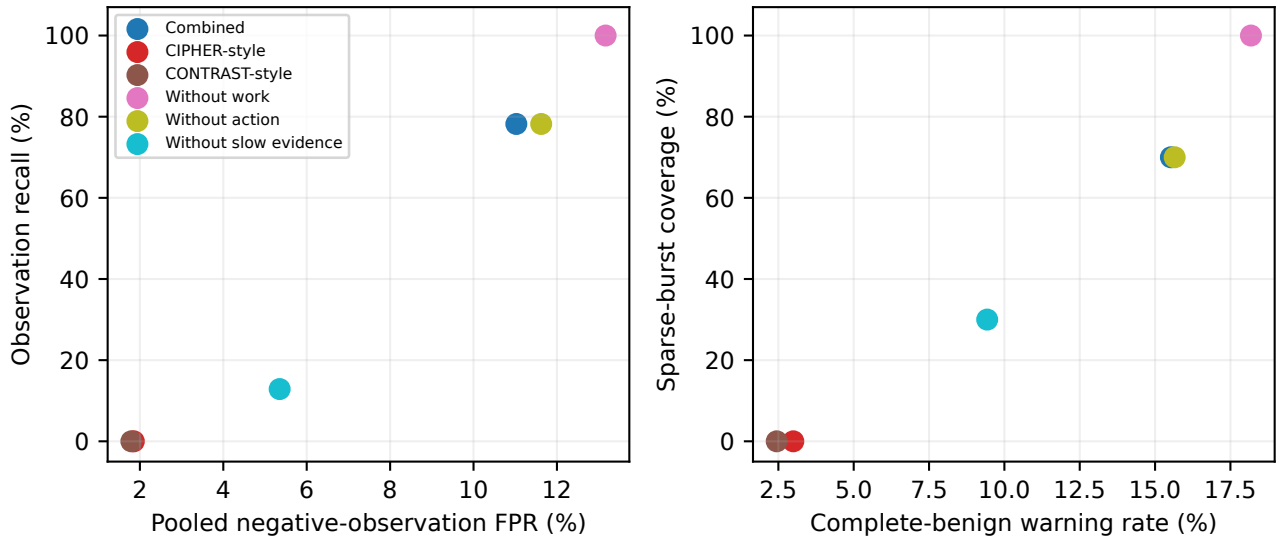


Fig. 3. Two operating-point views of the final Z1/gateway detector ablations. Left: pooled recall against pooled negative-observation FPR. Right: sparse-burst coverage against the complete-benign warning rate. These are frozen operating points, not ROC curves. Different denominators and pre-existing warning carryover prevent interpreting higher coverage alone as better discrimination.

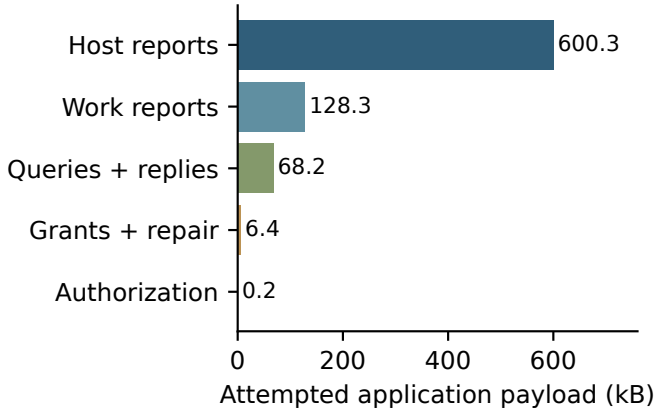


Fig. 4. Attempted defense application payload across the six full-model Z1/gateway runs. Host and work reports dominate; the 480 B/decision cap applies only to discretionary reservations. Derived from recorded channels, not measured energy.

leaf has no incoming application-UDP rows. Two ten-hour chain runs provide feasible placement and about 43,000 incoming UDP rows at the observer. The workload case executes 28,799 work events. The file case completes 2,459 CFS writes, totaling 9,836 bytes. We then reconstruct framework decisions in causal Python replay with 200-second cycles. Unavailable observations stay unknown. Collaboration counts are costs in the logical replay, not radio bytes measured in the network.

For SRF, we use an independent native Cooja port. We place 30 benign clients in the network. Attack runs add six malicious clients. We use 5/10/15 dedicated detectors, with all nodes in a 70×70 m area. Each detector setting has ten normal runs, ten SRF runs, and four malicious-MRHOF runs. Every run lasts one hour. Median all-client submitted PDR for SRF is 94.82%, 98.32%, and 96.48%. The corresponding MRHOF values are 93.12%, 92.39%, and 90.58%. Detector infrastructure remains present in all variants. This denominator includes malicious submissions and differs from our offered-work denominator. The SRF runs record 81, 93, and 125 false-blacklist diagnostics, respectively. A node may appear more than once in these diagnostics, so the counts do not represent distinct falsely accused nodes. We observe a response benefit alongside collateral effects in this port. Its scores are not directly comparable with those of the combined HMA controller.

VII. Lessons from the Wider Investigation

The broader research included analytic, synthetic, native, and emulated experiments. Their value is in the design decisions they support, not in adding every run count to the final study.

First, we used Python to enumerate candidate node subsets on path and star graphs. We asked whether the interpreted CIPHER placement cardinality rule alone ensured coverage. We found coverage holes in 15 of 70 examined path subsets. A star example had one internal node against a requested lower bound of five. These are counterexamples to our interpretation on abstract graphs. They do not demonstrate a failure in the authors' executable placement code. We did not retain placement as the paper's primary contribution.

Independent witnesses and adaptive sampling were not consistently better than simple counts. In the witness experiment (Section V-D), fixed and triggered observation each detected 11/16 episodes within 60 seconds. With authenticated real-data sampling, authenticated acknowledgments and source counts both reached 41/48. Their negative-observation rates differed: 6.04% and 0.50%, respectively. We measured these exploratory deadlines from nominal attack phases. The final study instead uses the first actual execution. The earlier results

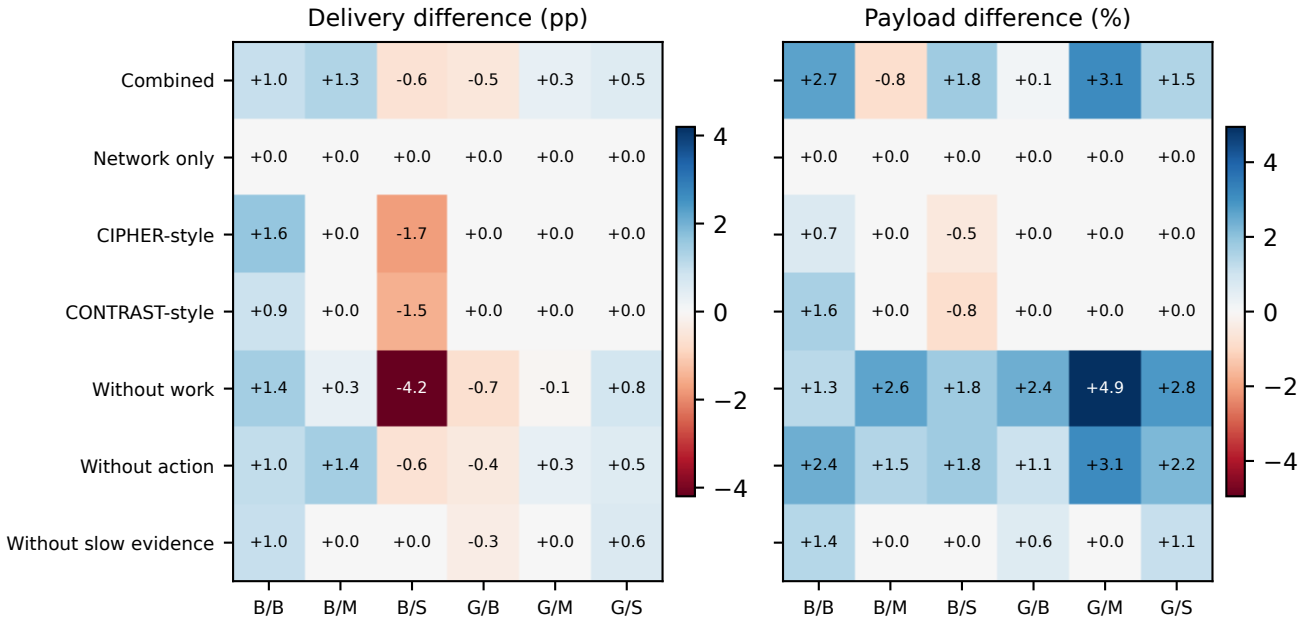


Fig. 5. Condition-wise differences from the equally instrumented network-only control, using all 42 final Z1/gateway runs. B/B, B/M, B/S denote bypass benign, mixed, and sparse/loss; G denotes grid. Each cell compares the same deployment/context. Positive delivery differences improve service; positive payload differences increase cost. Color scales are panel-specific.

guided the design; they do not estimate the final detector's accuracy.

We also screened adaptation and response choices in 24 native Cooja runs and a separate Python factorial model. The native runs tested source-trained detection on new grid/ladder conditions. In Python, we varied alternate delivery, confirmation after two observations, delayed benign-offset updates, and capped verification queues. Traffic and noise were paired across choices, giving 7,680 factorial and sensitivity records. The model assumed the accuracy of alternate delivery and confirmation; it did not measure them over radio. Sparse adaptation did not improve target detection. Some combinations that deferred decisions improved modelled service while leaving more malicious activity exposed. Those assumptions cannot validate the live controller. We kept unknown states explicit and limited investigation, without claiming a general open-set solution from delaying decisions.

Fourth, in the task-integrity experiment defined in Section V-D, we compared full checking with and without local completion of missing results. Fallback raised correct completions from 1,712/1,760 to 1,760/1,760 at 2.8% extra item computations. For this toy workload, central execution is an easy and strong alternative. The task verifier is not part of the final combined architecture, and task correctness does not prove absence of hidden host activity.

The standalone host experiment in Section V-D detected five of five fresh episodes. It issued no warnings among 8,239 scored benign observations, but covered only one of ten sparse bursts. Integration changed the predictor, traffic, and conditions. Benign warnings then became substantial. We therefore report the integrated result on its own; the earlier zero-warning rate does not transfer to it. That gap helped identify a remaining weakness in the model.

VIII. Limitations and Research Implications

Recording authorized work and issued responses changes which deficits the controller chooses to investigate. We implemented that link and measured lower communication cost in the native study. The benefit depends on the traffic and trust assumptions. We have not derived an optimal acquisition policy, and our comparisons cannot rule out all overlap with prior methods.

False warnings are the main obstacle to using the host detector. Precision of 21.61% and a 15.53% warning rate in benign runs do not support reliable HMA recognition. The integrated predictor must better handle forwarding load, workload-permission timing, and warning tails. A useful next comparison is a simple residual or linear model with the same workload information. That would test whether the GRU/classifier combination earns its extra complexity.

We tested the final model in two deployment blocks. Each topology has only one seed, so their effects are confounded. Repeated observations within a run do not increase the number of independent deployments. Some attack configurations also lack forwarding exposure. The broader model panels help identify transfer problems, but their different features and supervision prevent a single overall ranking.

A valid signature cannot make a false measurement true. Our forged-counter test fails when a compromised source signs false counters, and this implementation has no protected monitor. A separate Python fixture exposed a reset problem as well. After a counter discontinuity, the wrapper clears its state but the host detector inside it keeps its own state. The final

runs did not test resets. Reboot and counter-reset robustness require a correction followed by new validation.

Several deployment costs remain unmeasured. Gateway computation pauses the simulation, static RAM excludes peak stack/heap use, and payload totals omit PHY overhead and retransmissions. We have no hardware energy, timing, or FIT IoT-LAB measurements. Reducing report frequency may lower the dominant telemetry cost, but it also makes evidence older and can hide short bursts. That trade-off needs a fresh experiment.

IX. Conclusion

VIGILFOIL uses records of authorized work and repair to decide when more evidence is needed. It keeps original receipts separate from repairs and lowers the expected original count only when a supported redirect explains the change. In native Cooja, this accounting saves 17.97% of defense payload. The ten-Z1 integration gives a small gain in mean delivery, covers four host episodes, and covers seven of ten sparse bursts. It also produces substantial benign warnings and adds telemetry cost. The accounting benefit is measurable. Reliable host detection still requires better separation of benign activity and testing across more deployments.

References

[1] S. Sanyal, M. Khatua, and P. Chattopadhyay, "CIPHER: A collaborative ids placement and scheduling framework for host-oriented mimicry attack detection in iot networks," *IEEE Internet of Things Journal*, vol. 13, no. 14, pp. 31 880–31 888, 2026. [Online]. Available: https://doi.org/10.1109/JIOT.2026.3689273

[2] S. Sanyal, M. Khatua, and P. Chattopadhyay, "An Adaptive IDS Scheduling Framework for Host Based Mimicry Attack Detection in IoT Networks," in *2024 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, 2024, pp. 1–6. [Online]. Available: https://doi.org/10.1109/ants63515.2024.10898203

[3] S. Sanyal, M. Khatua, and P. Chattopadhyay, "LITMUS: A lightweight collaborative intrusion detection system for host oriented mimicry attack detection in industrial internet of things networks with dishonest collaborators in the majority," *Computer Networks*, p. 112474, 2026. [Online]. Available: https://doi.org/10.1016/j.comnet.2026.112474

[4] P. P. Ioulianou, V. G. Vassilakis, and S. F. Shahandashti, "A Trust-Based Intrusion Detection System for RPL Networks: Detecting a Combination of Rank and Blackhole Attacks," *Journal of Cybersecurity and Privacy*, pp. 124–153, 2022. [Online]. Available: https://doi.org/10.3390/jcp2010009

[5] P. P. Ioulianou, V. G. Vassilakis, and S. F. Shahandashti, "Archived SRF-IoT source release," Zenodo, 2022, archived release used for the declared Cooja port. [Online]. Available: https://doi.org/10.5281/zenodo.5828193

[6] W. Abbas, A. Laszka, Y. Vorobeychik, and X. Koutsoukos, "Scheduling intrusion detection systems in resource-bounded cyber-physical systems," in *Proceedings of the ACM Workshop on Cyber-Physical Systems Security and Privacy*, 2015, pp. 55–66. [Online]. Available: https://doi.org/10.1145/2808705.2808711

[7] T. Banerjee and V. V. Veeravalli, "Data-Efficient Quickest Change Detection in Minimax Settings," arXiv:1211.3729, 2012, preprint version. [Online]. Available: https://arxiv.org/abs/1211.3729

[8] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, "Recurrent Neural Networks for Multivariate Time Series with Missing Values," *Scientific Reports*, vol. 8, p. 6085, 2018. [Online]. Available: https://doi.org/10.1038/s41598-018-24271-9

[9] Y. Sun, A. Johnsson, and S. Banerjee, "Rethinking IoT Intrusion Detection: Augmenting Routing Metrics with Radio Features," arXiv:2606.07282, 2026, preprint; SNCNW 2026 acceptance reported by authors. [Online]. Available: https://arxiv.org/abs/2606.07282

[10] E. Lunderbye, S. Banerjee, C. Rohner, and A. Johnsson, "Towards Intrusion Detection Systems for RPL-based IoT Networks using Foundation Models," arXiv:2606.03530, 2026, preprint; SNCNW 2026 acceptance reported by authors. [Online]. Available: https://arxiv.org/abs/2606.03530

[11] MOMENT project contributors, "MOMENT: official implementation and pretrained MOMENT-1-small," 2024, weights: https://huggingface.co/AutonLab/MOMENT-1-small; frozen encoder reused in our benchmark. [Online]. Available: https://github.com/moment-timeseries-foundation-model/moment

[12] Uppsala University CORE research group, "IoT-Attacks-IDS: source code and released RPL datasets," 2026, study source pin: 8fa707f14c318495e7550fcdb410544cc0c4ad51. [Online]. Available: https://github.com/uu-core/IoT-Attacks-IDS

[13] T. Winter *et al.*, "RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks," Internet Engineering Task Force, Tech. Rep. RFC 6550, 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6550

[14] Contiki-NG contributors, "Cooja network simulator," simulator project reference; experiment-specific source and firmware pins are retained in the study manifests. [Online]. Available: https://github.com/contiki-ng/cooja